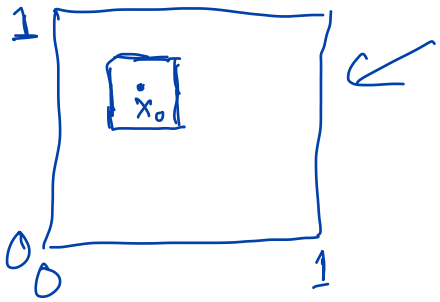
Multivariate Density Estimation

The idea of kernel density estimation extends naturally to d-dimensional data. Instead of a one-dimensional "bump" at each observation, the estimate is constructed from the sum of d-dimensional densities. The multivariate normal distribution is a standard choice.

Exercise: What are practical challenges to implementing this procedure?

1) How to choose smoothing parameter(s). In case where using multivariate normal as kernel, have a covariance matrix that defines the amount of smoothing. Not realistic to assume $\underset{\sim}{\Sigma} = h\underset{\sim}{I}$.

2) Curse of dimensionality - general nonparametric forms are difficult to estimate in high dimensions. More in Part 14.



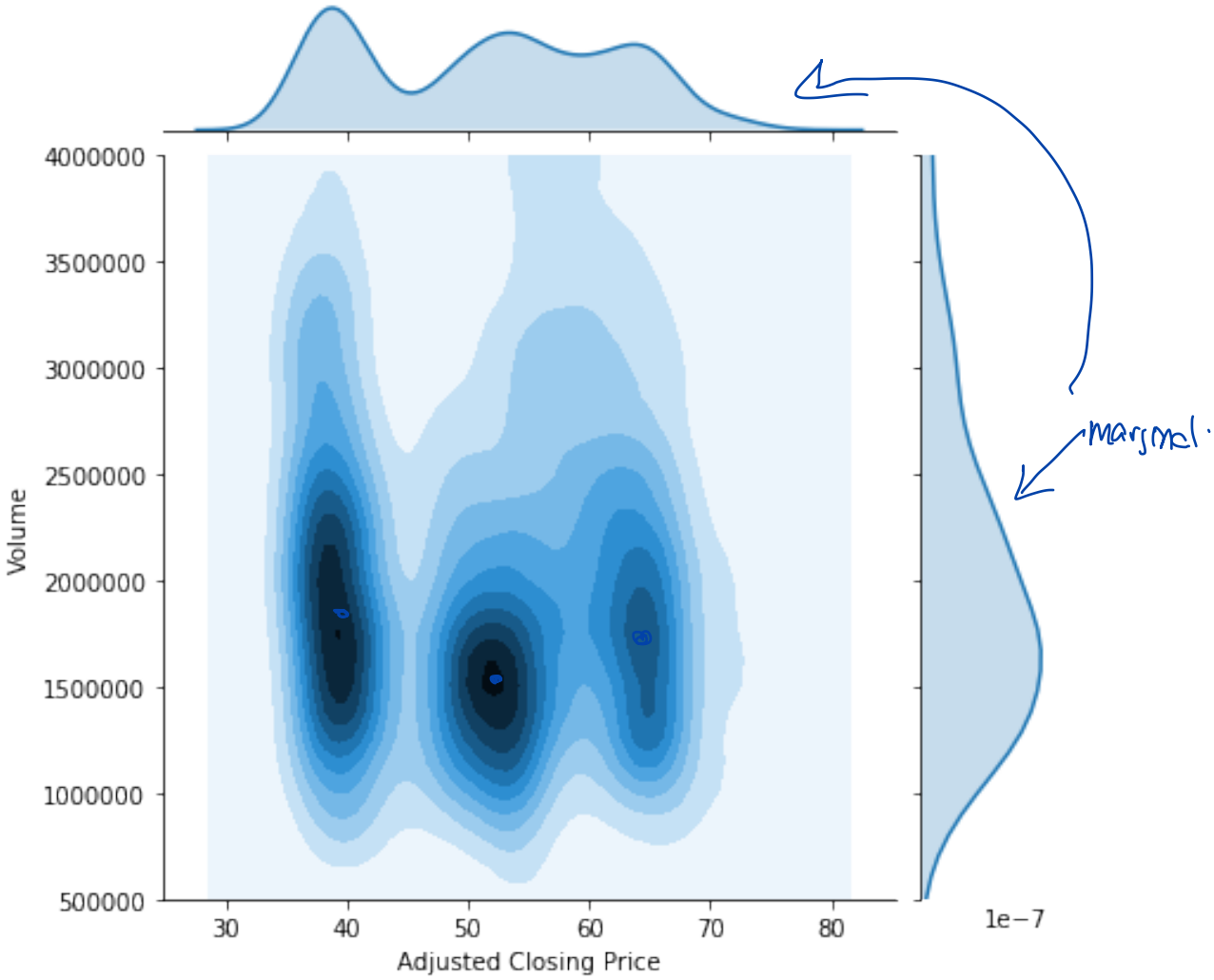
The function `FFTKDE()` works easily with d-dimensional data, although automatic bandwidth selection is not implemented.

```
In [11]: data = np.array(Kdat[['Adj Close','Volume']])
         np.array(data)

         kde = FFTKDE(kernel='gaussian')
         kdefit = kde.fit(data)
```

The `evaluate()` function is useful. For example, the syntax `kdefit.evaluate(10)` will evaluate the estimate at a regularly-spaced 10 by 10 grid of points.

Kernel density estimation is a crucial tool for data exploration and visualization. `Seaborn` has a number of built-in functions that directly construct kernel density estimates. See the following example.

```
In [12]: g = sns.jointplot(Kdat['Adj Close'],
            Kdat['Volume'], kind="kde",bw="silverman")
         g.set_axis_labels("Adjusted Closing Price",
            "Volume")
         plt.ylim(500000,4000000)
         plt.show()
```



See these other examples:

https://seaborn.pydata.org/examples/kde_ridgeplot.html

https://seaborn.pydata.org/examples/multiple_joint_kde.html

https://seaborn.pydata.org/examples/pair_grid_with_kde.html

Calculating SEs in general situations

General Setup:

$X_1, X_2, \ldots X_n \sim$ iid f_X

want to estimate $T(f_X)$, some function of f_X, e.g.

$T(f_X)$ = mean of dist.

$T(f_X)$ = median of dist.

$T(f_X)$ = tail prob. above c

One natural estimator for $T(f_X)$ is the "plug-in" estimator $T(\hat{f}_X)$, where $\hat{f}_X$ may be the result of fitting a parametric family via MLE or a nonparametric estimate of f_X, or the empirical dist.

In the case where we use the MLE to estimate f_X we know that the MLE for $T(f_X)$ is $T(\hat{f}_X)$? using the invariance property. We can use the Δ-method to approx. SEs for the estimator.

What about in other situations?

We have considered simulation-based approaches to estimating variance / SE of an estimator:

(1) For $b = 1, 2, \ldots, B$:

a) Draw $X_1^*, X_2^*, \ldots X_n^*$ from f_X

b) Construct estimate from new sample, obtain $\hat{T}_b^*$

② Calculate variance of $\hat{T}_1^*, \hat{T}_2^*, \ldots, \hat{T}_B^*$.
This approximates the variance of the estimator.

Of course, we don't know f_X. What to do?

Option 1: Make draws from $\hat{f}_X$.

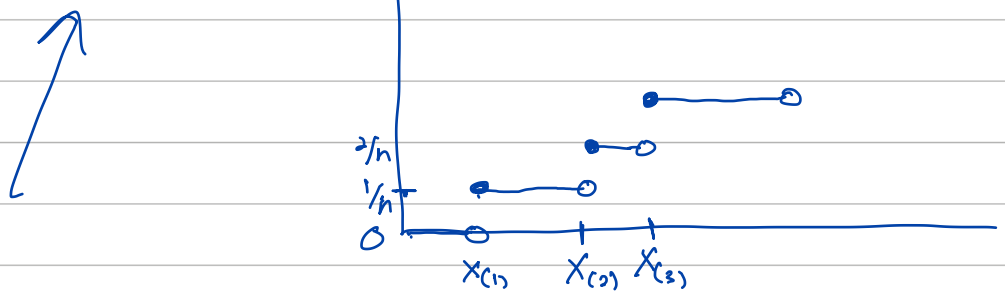
Option 2: Draw <u>with replacement</u> from $X_1, X_2, \ldots, X_n$

The "bootstrap."

The idea: Drawing with replacement from the observed data "mimics" drawing from the true dist.

Works very well. Very general.

<u>Empirical CDF</u>



<u>Example</u>

$X_1, X_2, \ldots, X_n$ iid f_X

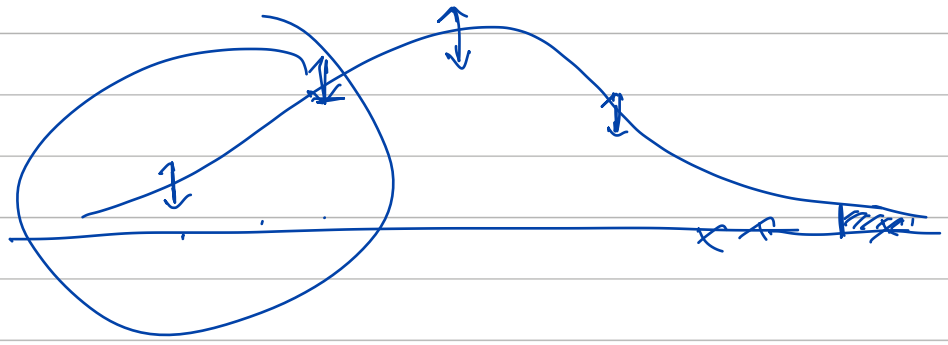
want to estimate 75th percentile

ie, $T(F_X) = 75^{th}$ percentile

our estimator is the sample 75^{th} percentile

What is its variance / SE ?

Use bootstrap.



Part 7: Smoothing Relationships

Smoothing refers generally to procedures that attempt to extract underlying **signal** from **noise** in a **nonparametric** manner.

The term "nonparametric" means the use of a statistical model that does not make restrictive assumptions. The data are allowed to "speak for themselves" as much as possible.

Smoothing techniques are very commonly applied to situations where one variable is naturally thought of as a **response**, i.e., a quantity to be predicted, and other variables are considered as **predictors**, i.e., the **regression** problem.

Example: Pricing Options

A **European call option** on a stock is a contract that gives the holder the right to purchase that stock at the named **strike price** on the **expiration date**.

A **American call option** is the same, but allows the holder to purchase the stock at that price **on or before** the expiration date.

The classic **Black-Scholes Theory** for option pricing establishes a price for a European option as a function of the following: the strike price, the time to expiration, the current asset price, the volatility of the price, and the current risk-free rate of return.

We will consider a data set consisting of 1,000 call options sampled on September 29, 2017.

To generate this sample, NYSE stock symbols were randomly chosen, and then one currently-listed call option was randomly chosen for each equity, weighted by the volume of trading.

Information recorded for each were as follows: The strike price, the time to expiration, the current asset price, the historical volatility (based on daily returns of 30 most recent trading days), the **implied volatility** and the Black-Scholes price for the option (with a risk-free rate of zero assumed).

This sample can be found in the Data Sets area on Canvas, with the name `optionssample09302017.txt`.

```
In [1]: import pandas as pd

        optionsdata = \
           pd.read_csv("optionssample09302017.txt", sep=",")
```

Our objective is to explore the relationships between the current ask price for the option and the available variables.

This is an "empirical" alternative to Black-Scholes theory, whose limitations are well-known, mainly due to the failure of the underlying normality assumption.

A classic manifestation of the failure of Black-Scholes is the so-called **volatility smile**. Under the theory, a plot of implied volatility versus how far "in the money" the option currently is should be flat for options with the same expiration date.

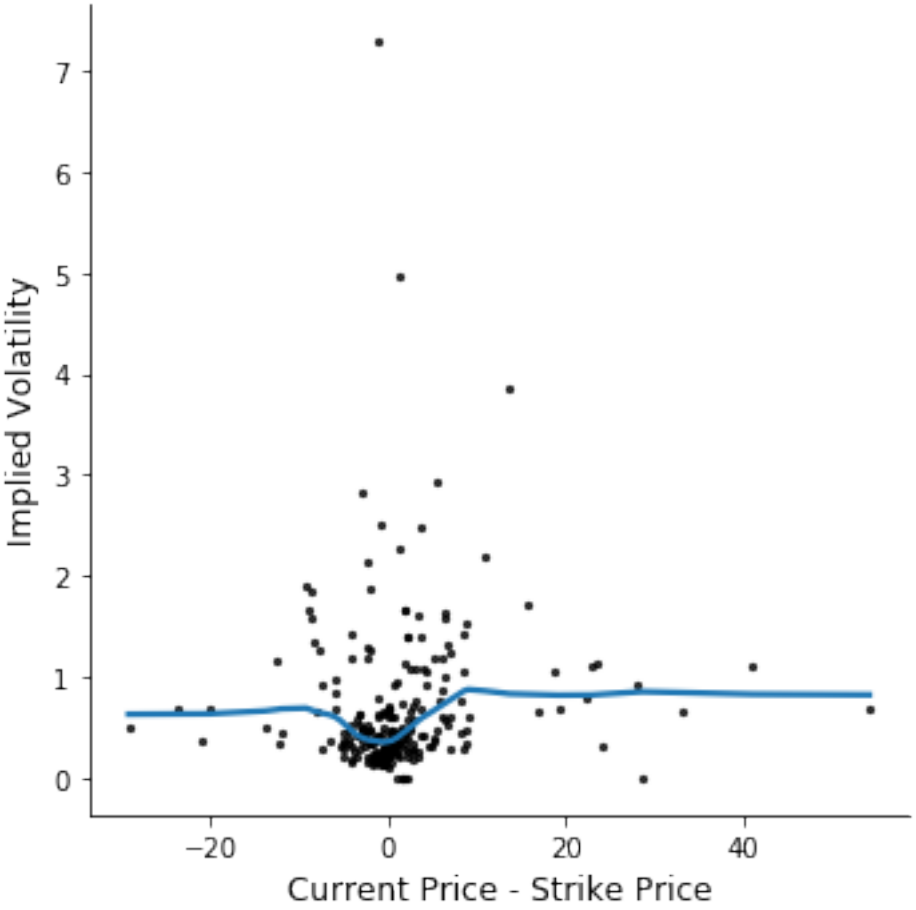
Let's create this plot in our case. Is there evidence of the smile here?

```
In [2]: optionsdata["inmoney"] = \
          optionsdata["curprice"] - optionsdata["strike"]

        optionsdata20 = \
          optionsdata[optionsdata["timetoexpiry"] == 20]

        import seaborn as sns
        import matplotlib.pyplot as plt

        ax = sns.scatterplot(x="inmoney", y="implvol",
              data=optionsdata20, s=5, color="black")
        plt.xlabel("Current Price - Strike Price", size=12)
        plt.ylabel("Implied Volatility", size=12)
        plt.show()
```

```
<Figure size 640x480 with 1 Axes>
```

The seaborn function `lmplot()` allows for the inclusion of a smoothed version of the relationship between the variables. This can be done by setting `lowess = True`.

```
In [3]: sns.lmplot(x="inmoney", y="implvol",
                   data=optionsdata20, lowess=True,
                   scatter_kws={"s": 5, "color": "black"})
        plt.xlabel("Current Price - Strike Price",size=12)
        plt.ylabel("Implied Volatility",size=12)
        plt.show()
```



Exercise: Comment on the addition of the smooth to the plot.